You can order print and ebook versions of *Think Bayes 2e* from Bookshop.org and Amazon.

# MCMC

[NOTE: this online version of Chapter 19 has been updated for PyMC version 5]

For most of this book we've used grid methods to approximate posterior distributions. For models with one or two parameters, grid algorithms are fast and the results are precise enough for most practical purposes. With three parameters, they start to be slow, and with more than three they are usually not practical.

In the previous chapter we saw that we can solve some problems using conjugate priors. But the problems we can solve this way tend to be the same ones we can solve with grid algorithms.

For problems with more than a few parameters, the most powerful tool we have is MCMC, which stands for "Markov chain Monte Carlo". In this context, "Monte Carlo" refers to methods that generate random samples from a distribution. Unlike grid methods, MCMC methods don't try to compute the posterior distribution; they sample from it instead.

It might seem strange that you can generate a sample without ever computing the distribution, but that's the magic of MCMC.

To demonstrate, we'll start by solving the World Cup problem. Yes, again.

```
# install empiricaldist if necessary

try:
    import empiricaldist
except ImportError:
    !pip install empiricaldist
    import empiricaldist
```

```
# Get utils.py

from os.path import basename, exists

def download(url):
    filename = basename(url)
    if not exists(filename):
        from urllib.request import urlretrieve
        local, _ = urlretrieve(url, filename)
        print('Downloaded ' + local)

download('https://github.com/AllenDowney/ThinkBayes2/raw/master/soln/utils.py')
```

```
from utils import set_pyplot_params
set_pyplot_params()
```

## The World Cup Problem

In <<_PoissonProcesses>> we modeled goal scoring in football (soccer) as a Poisson process characterized by a goal-scoring rate, denoted $\lambda$.

We used a gamma distribution to represent the prior distribution of $\lambda$, then we used the outcome of the game to compute the posterior distribution for both teams.

To answer the first question, we used the posterior distributions to compute the "probability of superiority" for France.

To answer the second question, we computed the posterior predictive distributions for each team, that is, the distribution of goals we expect in a rematch.

In this chapter we'll solve this problem again using PyMC, which is a library that provide implementations of several MCMC methods. But we'll start by reviewing the grid approximation of the prior and the prior predictive distribution.

## Grid Approximation

As we did in <<_TheGammaDistribution>> we'll use a gamma distribution with parameter $\alpha = 1.4$ to represent the prior.

```
from scipy.stats import gamma

alpha = 1.4
prior_dist = gamma(alpha)
```

I'll use `linspace` to generate possible values for $\lambda$, and `pmf_from_dist` to compute a discrete approximation of the prior.

```
import numpy as np
from utils import pmf_from_dist

lams = np.linspace(0, 10, 101)
prior_pmf = pmf_from_dist(prior_dist, lams)
```

We can use the Poisson distribution to compute the likelihood of the data; as an example, we'll use 4 goals.

```
from scipy.stats import poisson

data = 4
likelihood = poisson.pmf(data, lams)
```

Now we can do the update in the usual way.

```
posterior = prior_pmf * likelihood
posterior.normalize()
```

```
np.float64(0.05015532557804499)
```

Soon we will solve the same problem with PyMC, but first it will be useful to introduce something new: the prior predictive distribution.

## Prior Predictive Distribution

We have seen the posterior predictive distribution in previous chapters; the prior predictive distribution is similar except that (as you might have guessed) it is based on the prior.

To estimate the prior predictive distribution, we'll start by drawing a sample from the prior.

```
sample_prior = prior_dist.rvs(1000)
```

The result is an array of possible values for the goal-scoring rate, $\lambda$. For each value in `sample_prior`, I'll generate one value from a Poisson distribution.

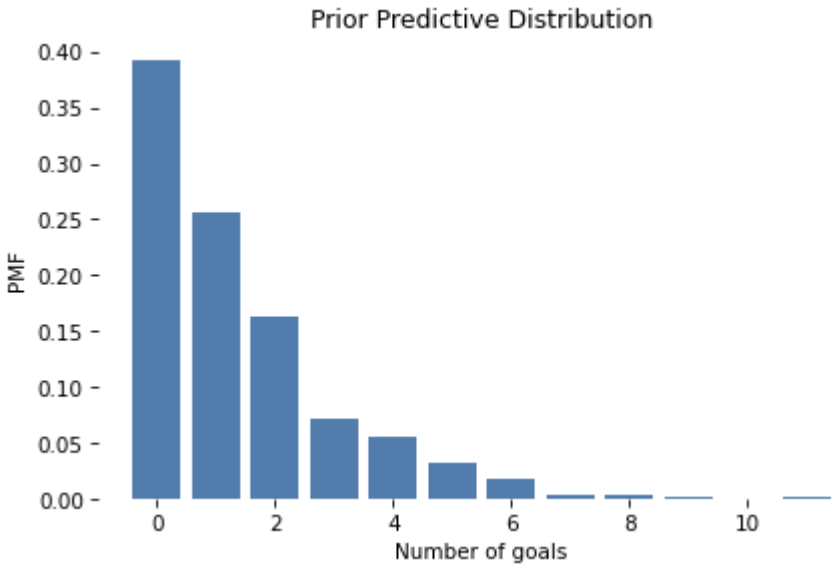
```
from scipy.stats import poisson

sample_prior_pred = poisson.rvs(sample_prior)
```

`sample_prior_pred` is a sample from the prior predictive distribution. To see what it looks like, we'll compute the PMF of the sample.

```
from empiricaldist import Pmf

pmf_prior_pred = Pmf.from_seq(sample_prior_pred)
```

And here's what it looks like:

```
from utils import decorate

pmf_prior_pred.bar()
decorate(xlabel='Number of goals',
         ylabel='PMF',
         title='Prior Predictive Distribution')
```




One reason to compute the prior predictive distribution is to check whether our model of the system seems reasonable. In this case, the distribution of goals seems consistent with what we know about World Cup football.

But in this chapter we have another reason: computing the prior predictive distribution is a first step toward using MCMC.

## Introducing PyMC

PyMC is a Python library that provides several MCMC methods. To use PyMC, we have to specify a model of the process that generates the data. In this example, the model has two steps:

- First we draw a goal-scoring rate from the prior distribution,
- Then we draw a number of goals from a Poisson distribution.

Here's how we specify this model in PyMC:

```
import pymc as pm

with pm.Model() as model:
    lam = pm.Gamma('lam', alpha=1.4, beta=1.0)
    goals = pm.Poisson('goals', lam)
```

After importing `PyMC`, we create a `Model` object named `model`.

If you are not familiar with the `with` statement in Python, it is a way to associate a block of statements with an object. In this example, the two indented statements are associated with the new `Model` object. As a result, when we create the distribution objects, `Gamma` and `Poisson`, they are added to the `Model`.

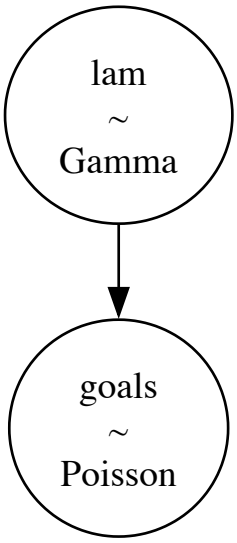
Inside the `with` statement:

- The first line creates the prior, which is a gamma distribution with the given parameters.
- The second line creates the prior predictive, which is a Poisson distribution with the parameter `lam`.

The first parameter of `Gamma` and `Poisson` is a string variable name.

PyMC provides a function that generates a visual representation of the model.

```
pm.model_to_graphviz(model)
```




In this visualization, the ovals show that `lam` is drawn from a gamma distribution and `goals` is drawn from a Poisson distribution. The arrow shows that the values of `lam` are used as parameters for the distribution of `goals`.

## Sampling the Prior

PyMC provides a function that generates samples from the prior and prior predictive distributions. We can use a `with` statement to run this function in the context of the model.

```
with model:
    idata = pm.sample_prior_predictive(1000)
```

```
Sampling: [goals, lam]
```

```
type(idata)
```

```
arviz.data.inference_data.InferenceData
```

The result is an `InferenceData` object that contains information about the sampling process and the results. We can extract the sample of `lam` like this:

```
def get_values(array):
    return array.values.flatten()
```

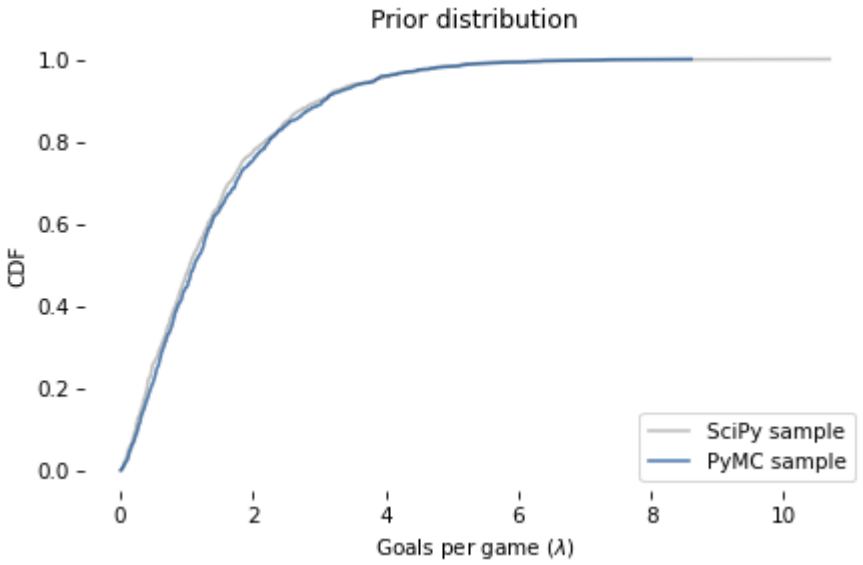
```
sample_prior_pymc = get_values(idata.prior['lam'])
sample_prior_pymc.shape
```

```
(1000,)
```

The following figure compares the CDF of this sample to the CDF of the sample we generated using the `gamma` object from SciPy.

```
from empiricaldist import Cdf

def plot_cdf(sample, **options):
    """Plot the CDF of a sample.

    sample: sequence of quantities
    """
    Cdf.from_seq(sample).plot(**options)
```

```
plot_cdf(sample_prior,
         label='SciPy sample',
         color='C5')
plot_cdf(sample_prior_pymc,
         label='PyMC sample',
         color='C0')
decorate(xlabel=r'Goals per game ($\lambda$)',
         ylabel='CDF',
         title='Prior distribution')
```




The results are similar, which confirms that the specification of the model is correct and the sampler works as advertised.

From the inference data we can also extract `goals`, which is a sample from the prior predictive distribution.

```
sample_prior_pred_pymc = get_values(idata.prior['goals'])
sample_prior_pred_pymc.shape
```

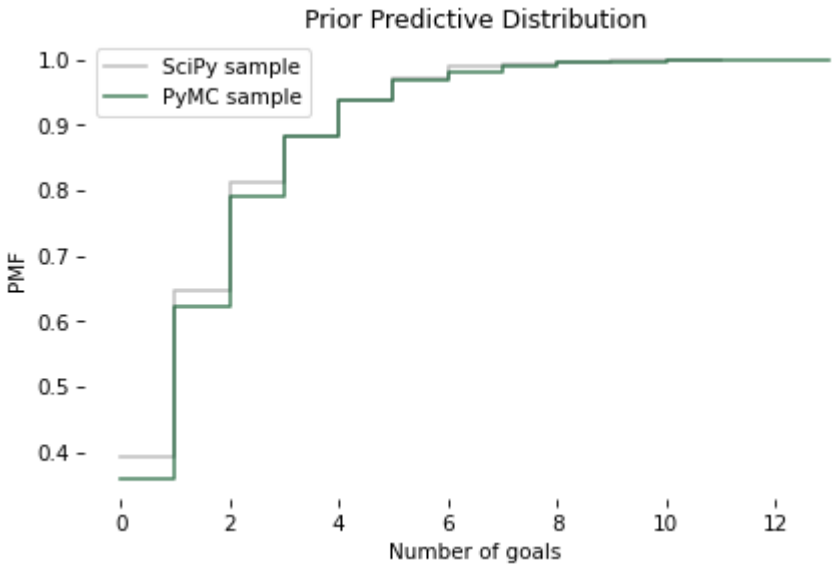
```
(1000,)
```

And we can compare it to the sample we generated using the `poisson` object from SciPy.

Because the quantities in the posterior predictive distribution are discrete (number of goals) I'll plot the CDFs as step functions.

```
def plot_pred(sample, **options):
    Cdf.from_seq(sample).step(**options)
```

```
plot_pred(sample_prior_pred,
          label='SciPy sample',
          color='C5')
plot_pred(sample_prior_pred_pymc,
          label='PyMC sample',
          color='C13')
decorate(xlabel='Number of goals',
         ylabel='PMF',
         title='Prior Predictive Distribution')
```




Again, the results are similar, so we have some confidence we are using PyMC right.

## When Do We Get to Inference?

Finally, we are ready for actual inference. We just have to make one small change. Here is the model we used to generate the prior predictive distribution:

```
with pm.Model() as model:
    lam = pm.Gamma('lam', alpha=1.4, beta=1.0)
    goals = pm.Poisson('goals', lam)
```

And here is the model we'll use to compute the posterior distribution.

```
with pm.Model() as model2:
    lam = pm.Gamma('lam', alpha=1.4, beta=1.0)
    goals = pm.Poisson('goals', lam, observed=4)
```

The difference is that we mark goals as `observed` and provide the observed data, `4`.

And instead of calling `sample_prior_predictive`, we'll call `sample`, which is understood to sample from the posterior distribution of `lam`.

```
options = dict()

with model2:
    idata2 = pm.sample(500, **options)
```

```
Initializing NUTS using jitter+adapt_diag...
```

```
Multiprocess sampling (4 chains in 4 jobs)
```

```
NUTS: [lam]
```

```
/Users/anton/.pyenv/versions/3.14.3/lib/python3.14/site-packages/rich/live.py:260: UserWarning:
"ipywidgets" for Jupyter support
  warnings.warn('install "ipywidgets" for Jupyter support')
```

```
Sampling 4 chains for 1_000 tune and 500 draw iterations (4_000 + 2_000 draws total) took 0 sec
onds.
```

Although the specification of these models is similar, the sampling process is very different. I won't go into the details of how PyMC works, but here are a few things you should be aware of:

- Depending on the model, PyMC uses one of several MCMC methods; in this example, it uses the No U-Turn Sampler (NUTS), which is one of the most efficient and reliable methods we have.
- When the sampler starts, the first values it generates are usually not a representative sample from the posterior distribution, so these values are discarded. This process is called "tuning".
- Instead of using a single Markov chain, PyMC uses multiple chains. Then we can compare results from multiple chains to make sure they are consistent.

Although we asked for a sample of 500, PyMC generated two samples of 1000, discarded half of each, and returned the remaining 1000. From `idata2` we can extract a sample from the posterior distribution, like this:

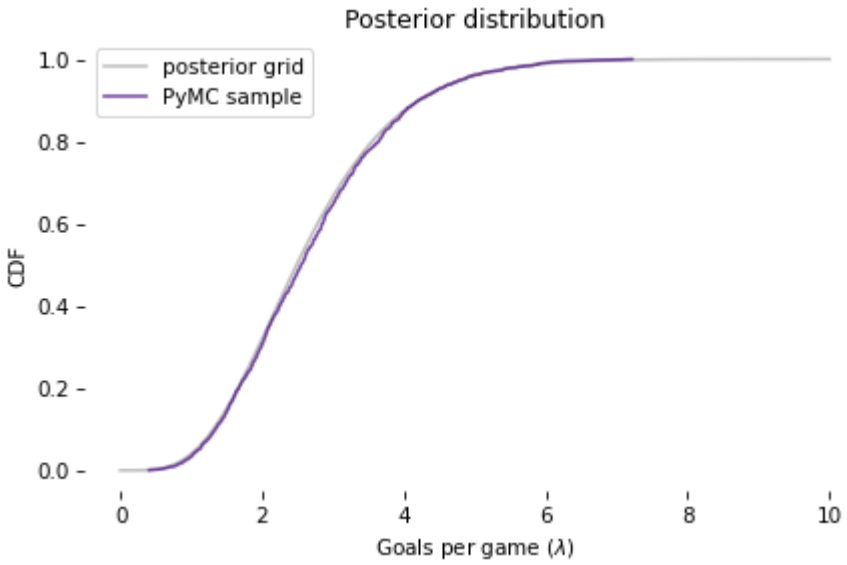
```
sample_post_pymc = get_values(idata2.posterior['lam'])
```

```
sample_post_pymc.shape
```

```
(2000,)
```

And we can compare the CDF of this sample to the posterior we computed by grid approximation:

```
posterior.make_cdf().plot(label='posterior grid',
                          color='C5')
plot_cdf(sample_post_pymc,
         label='PyMC sample',
         color='C4')

decorate(xlabel=r'Goals per game ($\lambda$)',
         ylabel='CDF',
         title='Posterior distribution')
```




The results from PyMC are consistent with the results from the grid approximation.

## Posterior Predictive Distribution

Finally, to sample from the posterior predictive distribution, we can use `sample_posterior_predictive`:

```
with model2:
    idata2_pred = pm.sample_posterior_predictive(idata2)
```

```
Sampling: [goals]
```

```
/Users/anton/.pyenv/versions/3.14.3/lib/python3.14/site-packages/rich/live.py:260: UserWarning:
"ipywidgets" for Jupyter support
  warnings.warn('install "ipywidgets" for Jupyter support')
```

The result is an `InferenceData` object that contains a sample of `goals`.

```
sample_post_pred_pymc = get_values(idata2_pred.posterior_predictive['goals'])
```

```
sample_post_pred_pymc.shape
```

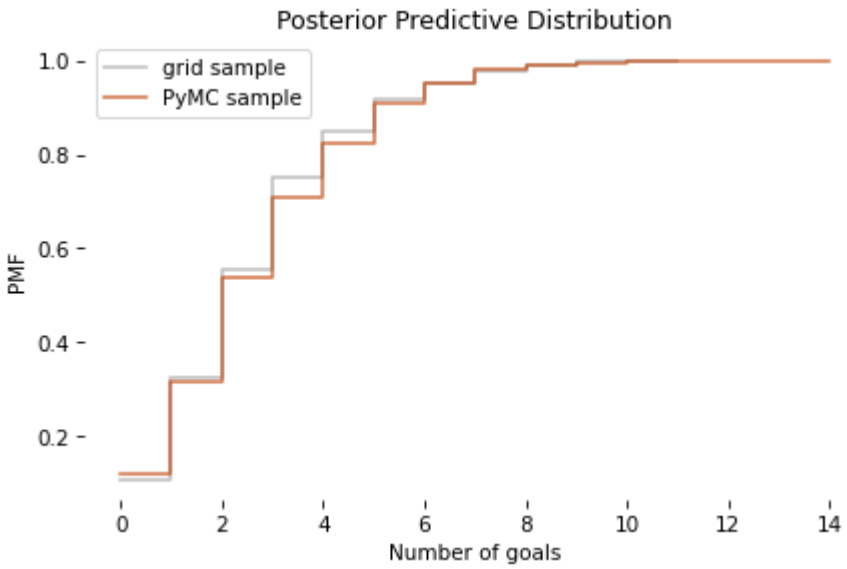
```
(2000,)
```

I'll also generate a sample from the posterior distribution we computed by grid approximation.

```
sample_post = posterior.sample(1000)
sample_post_pred = poisson(sample_post).rvs()
```

And we can compare the two samples.

```
plot_pred(sample_post_pred,
          label='grid sample',
          color='C5')
plot_pred(sample_post_pred_pymc,
          label='PyMC sample',
          color='C12')

decorate(xlabel='Number of goals',
         ylabel='PMF',
         title='Posterior Predictive Distribution')
```




Again, the results are consistent. So we've established that we can compute the same results using a grid approximation or PyMC.

But it might not be clear why. In this example, the grid algorithm requires less computation than MCMC, and the result is a pretty good approximation of the posterior distribution, rather than a sample.

However, this is a simple model with just one parameter. In fact, we could have solved it with even less computation, using a conjugate prior. The power of PyMC will be clearer with a more complex model.

## Happiness

Recently I read "Happiness and Life Satisfaction" by Esteban Ortiz-Ospina and Max Roser, which discusses (among many other things) the relationship between income and happiness, both between countries, within countries, and over time.

It cites the "World Happiness Report", which includes results of a multiple regression analysis that explores the relationship between happiness and six potentially predictive factors:

- Income as represented by per capita GDP
- Social support
- Healthy life expectancy at birth
- Freedom to make life choices
- Generosity
- Perceptions of corruption

The dependent variable is the national average of responses to the "Cantril ladder question" used by the Gallup World Poll:

> *Please imagine a ladder with steps numbered from zero at the bottom to 10 at the top. The top of the ladder represents the best possible life for you and the bottom of the ladder represents the worst possible life for you. On which step of the ladder would you say you personally feel you stand at this time?*

I'll refer to the responses as "happiness", but it might be more precise to think of them as a measure of satisfaction with quality of life.

In the next few sections we'll replicate the analysis in this report using Bayesian regression.

The data from this report can be downloaded from here.

```
# Get the data file

download('https://happiness-report.s3.amazonaws.com/2020/WHR20_DataForFigure2.1.xls')
```

We can use Pandas to read the data into a `DataFrame`.

```
import pandas as pd

filename = 'WHR20_DataForFigure2.1.xls'
df = pd.read_excel(filename)
```

```
df.head(3)
```

| | Country name | Regional indicator | Ladder score | Standard error of ladder score | upperwhisker | lowerv |
|---|---|---|---|---|---|---|
| 0 | Finland | Western Europe | 7.8087 | 0.031156 | 7.869766 | 7 |
| 1 | Denmark | Western Europe | 7.6456 | 0.033492 | 7.711245 | 7 |
| 2 | Switzerland | Western Europe | 7.5599 | 0.035014 | 7.628528 | 7 |

```
df.shape
```

```
(153, 20)
```

The `DataFrame` has one row for each of 153 countries and one column for each of 20 variables.

The column called `'Ladder score'` contains the measurements of happiness we will try to predict.

```
score = df['Ladder score']
```

## Simple Regression

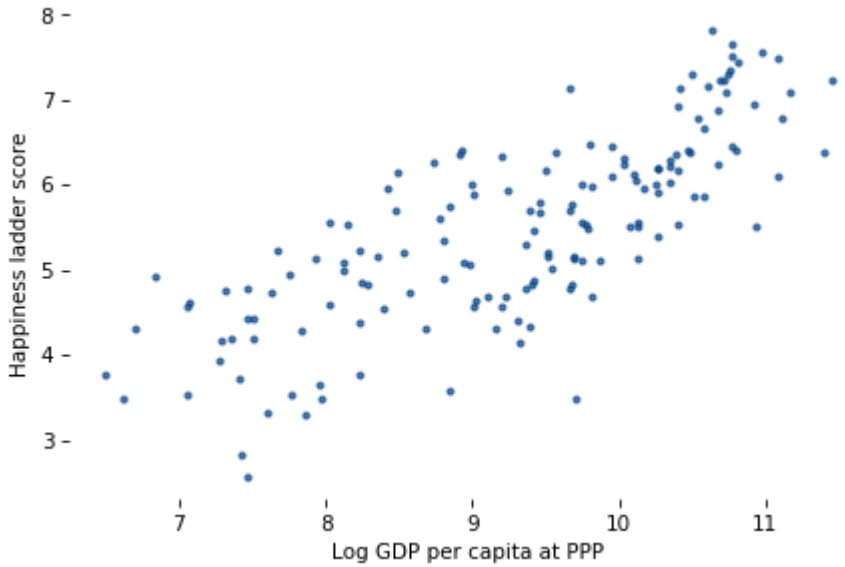
To get started, let's look at the relationship between happiness and income as represented by gross domestic product (GDP) per person.

The column named `'Logged GDP per capita'` represents the natural logarithm of GDP for each country, divided by population, corrected for purchasing power parity (PPP).

```
log_gdp = df['Logged GDP per capita']
```

The following figure is a scatter plot of `score` versus `log_gdp`, with one marker for each country.

```
import matplotlib.pyplot as plt

plt.plot(log_gdp, score, '.')

decorate(xlabel='Log GDP per capita at PPP',
         ylabel='Happiness ladder score')
```




It's clear that there is a relationship between these variables: people in countries with higher GDP generally report higher levels of happiness.

We can use `linregress` from SciPy to compute a simple regression of these variables.

```
from scipy.stats import linregress

result = linregress(log_gdp, score)
```

And here are the results.

```
pd.DataFrame([result.slope, result.intercept],
             index=['Slope', 'Intercept'],
             columns=[''])
```

| | |
|---|---|
| **Slope** | 0.717738 |
| **Intercept** | -1.198646 |

The estimated slope is about 0.72, which suggests that an increase of one unit in log-GDP, which is a factor of $e \approx 2.7$ in GDP, is associated with an increase of 0.72 units on the happiness ladder.

Now let's estimate the same parameters using PyMC. We'll use the same regression model as in Section <<_RegressionModel>>:

$$y = ax + b + \epsilon$$

where $y$ is the dependent variable (ladder score), $x$ is the predictive variable (log GDP) and $\epsilon$ is a series of values from a normal distribution with standard deviation $\sigma$.

$a$ and $b$ are the slope and intercept of the regression line. They are unknown parameters, so we will use the data to estimate them.

The following is the PyMC specification of this model.

```
x_data = log_gdp
y_data = score

with pm.Model() as model3:
    a = pm.Uniform('a', 0, 4)
    b = pm.Uniform('b', -4, 4)
    sigma = pm.Uniform('sigma', 0, 2)

    y_est = a * x_data + b
    y = pm.Normal('y',
                  mu=y_est, sigma=sigma,
                  observed=y_data)
```

The prior distributions for the parameters `a`, `b`, and `sigma` are uniform with ranges that are wide enough to cover the posterior distributions.

`y_est` is the estimated value of the dependent variable, based on the regression equation. And `y` is a normal distribution with mean `y_est` and standard deviation `sigma`.

Notice how the data are included in the model:

- The values of the predictive variable, `x_data`, are used to compute `y_est`.
- The values of the dependent variable, `y_data`, are provided as the observed values of `y`.

Now we can use this model to generate a sample from the posterior distribution.

```
with model3:
    idata3 = pm.sample(500, **options)
```

```
Initializing NUTS using jitter+adapt_diag...
```

```
Multiprocess sampling (4 chains in 4 jobs)
```

```
NUTS: [a, b, sigma]
```

```
/Users/anton/.pyenv/versions/3.14.3/lib/python3.14/site-packages/rich/live.py:260: UserWarning:
"ipywidgets" for Jupyter support
  warnings.warn('install "ipywidgets" for Jupyter support')
```

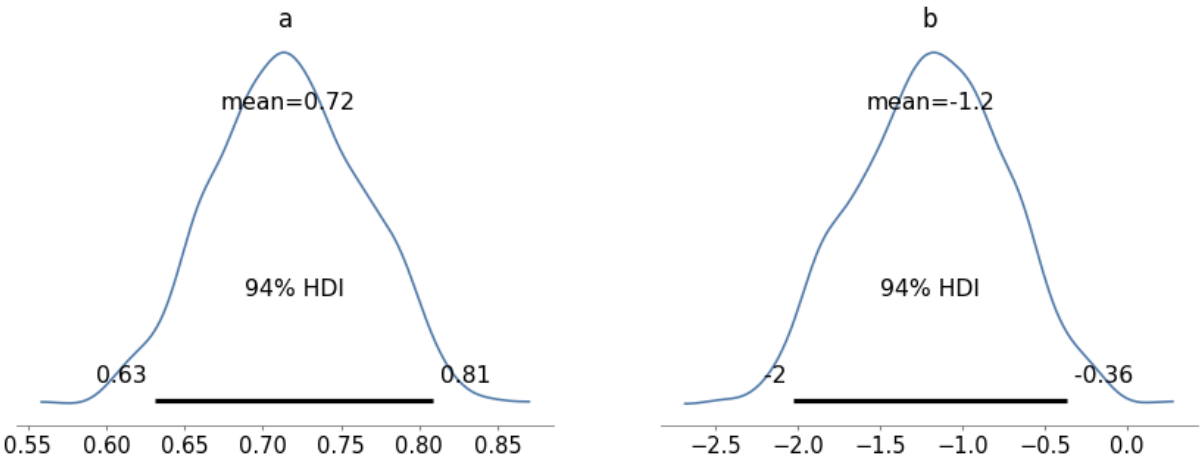
```
Sampling 4 chains for 1_000 tune and 500 draw iterations (4_000 + 2_000 draws total) took 1 sec
onds.
```

When you run the sampler, you might get warning messages about "divergences" and the "acceptance probability". You can ignore them for now.

The result is an object that contains samples from the joint posterior distribution of `a`, `b`, and `sigma`.

ArviZ provides `plot_posterior`, which we can use to plot the posterior distributions of the parameters. Here are the posterior distributions of slope, `a`, and intercept, `b`.

```
import arviz as az

with model3:
    az.plot_posterior(idata3, var_names=['a', 'b']);
```




The graphs show the distributions of the samples, estimated by KDE, and 94% credible intervals. In the figure, "HDI" stands for "highest-density interval".

The means of these samples are consistent with the parameters we estimated with `linregress`.

```
print('Sample mean:', get_values(idata3.posterior['a'].mean()))
print('Regression slope:', result.slope)
```

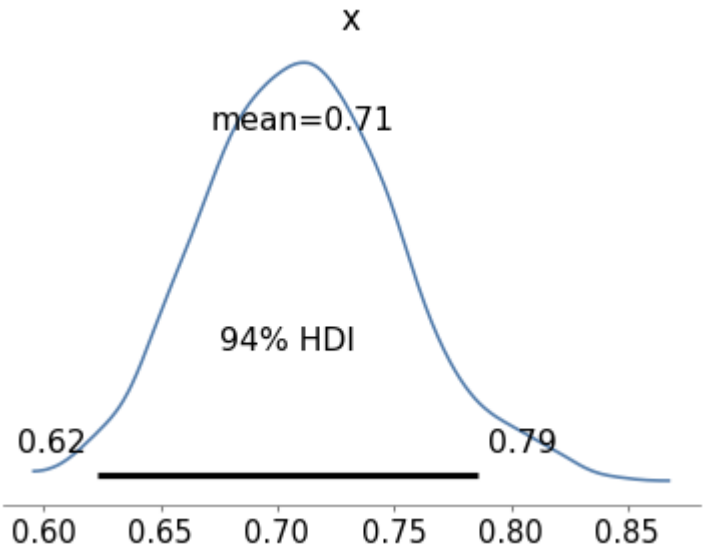
```
Sample mean: [0.71649428]
Regression slope: 0.7177384956304522
```

```
print('Sample mean:', get_values(idata3.posterior['b'].mean()))
print('Regression intercept:', result.intercept)
```

```
Sample mean: [-1.18795689]
Regression intercept: -1.198646061808887
```

Finally, we can check the marginal posterior distribution of `sigma`

```
az.plot_posterior(get_values(idata3.posterior['sigma']));
```




The values in the posterior distribution of `sigma` seem plausible.

The simple regression model has only three parameters, so we could have used a grid algorithm. But the regression model in the happiness report has six predictive variables, so it has eight parameters in total, including the intercept and `sigma`.

It is not practical to compute a grid approximation for a model with eight parameters. Even a coarse grid, with 20 points along each dimension, would have more than 25 billion points. And with 153 countries, we would have to compute almost 4 trillion likelihoods.

But PyMC can handle a model with eight parameters comfortably, as we'll see in the next section.

```
20 ** 8 / 1e9
```

```
25.6
```

```
153 * 20 ** 8 / 1e12
```

```
3.9168
```

## Multiple Regression

Before we implement the multiple regression model, I'll select the columns we need from the `DataFrame`.

```
columns = ['Ladder score',
           'Logged GDP per capita',
           'Social support',
           'Healthy life expectancy',
           'Freedom to make life choices',
           'Generosity',
           'Perceptions of corruption']

subset = df[columns]
```

```
subset.head(3)
```

| | Ladder score | Logged GDP per capita | Social support | Healthy life expectancy | Freedom to make life choices | Generosity |
|---|---|---|---|---|---|---|
| 0 | 7.8087 | 10.639267 | 0.954330 | 71.900826 | 0.949172 | -0.059482 |
| 1 | 7.6456 | 10.774001 | 0.955991 | 72.402504 | 0.951444 | 0.066202 |
| 2 | 7.5599 | 10.979933 | 0.942847 | 74.102448 | 0.921337 | 0.105911 |

The predictive variables have different units: log-GDP is in log-dollars, life expectancy is in years, and the other variables are on arbitrary scales. To make these factors comparable, I'll standardize the data so that each variable has mean 0 and standard deviation 1.

```
standardized = (subset - subset.mean()) / subset.std()
```

Now let's build the model. I'll extract the dependent variable.

```
y_data = standardized['Ladder score']
```

And the dependent variables.

```
x1 = standardized[columns[1]]
x2 = standardized[columns[2]]
x3 = standardized[columns[3]]
x4 = standardized[columns[4]]
x5 = standardized[columns[5]]
x6 = standardized[columns[6]]
```

And here's the model. `b0` is the intercept; `b1` through `b6` are the parameters associated with the predictive variables.

```
with pm.Model() as model4:
    b0 = pm.Uniform('b0', -4, 4)
    b1 = pm.Uniform('b1', -4, 4)
    b2 = pm.Uniform('b2', -4, 4)
    b3 = pm.Uniform('b3', -4, 4)
    b4 = pm.Uniform('b4', -4, 4)
    b5 = pm.Uniform('b5', -4, 4)
    b6 = pm.Uniform('b6', -4, 4)
    sigma = pm.Uniform('sigma', 0, 2)

    y_est = b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 + b5*x5 + b6*x6
    y = pm.Normal('y',
                  mu=y_est, sigma=sigma,
                  observed=y_data)
```

We could express this model more concisely using a vector of predictive variables and a vector of parameters, but I decided to keep it simple.

Now we can sample from the joint posterior distribution.

```
with model4:
    idata4 = pm.sample(500, **options)
```

```
Initializing NUTS using jitter+adapt_diag...
```

```
Multiprocess sampling (4 chains in 4 jobs)
```

```
NUTS: [b0, b1, b2, b3, b4, b5, b6, sigma]
```

```
/Users/anton/.pyenv/versions/3.14.3/lib/python3.14/site-packages/rich/live.py:260: UserWarning:
"ipywidgets" for Jupyter support
  warnings.warn('install "ipywidgets" for Jupyter support')
```

```
Sampling 4 chains for 1_000 tune and 500 draw iterations (4_000 + 2_000 draws total) took 1 sec
onds.
```

Because we standardized the data, we expect the intercept to be 0, and in fact the posterior mean of `b0` is close to 0.

```
get_values(idata4.posterior['b0']).mean()
```

```
np.float64(0.0002344469870419541)
```

We can also check the posterior mean of `sigma`:

```
get_values(idata4.posterior['sigma']).mean()
```

```
np.float64(0.5151927308337183)
```

From `idata4` we can extract samples from the posterior distributions of the parameters and compute their means.

```
param_names = ['b1', 'b3', 'b3', 'b4', 'b5', 'b6']

means = [get_values(idata4.posterior[name]).mean()
         for name in param_names]
```

We can also compute 94% credible intervals (between the 3rd and 97th percentiles).

```
def credible_interval(sample):
    """Compute 94% credible interval."""
    ci = np.percentile(sample, [3, 97])
    return np.round(ci, 3)

cis = [credible_interval(get_values(idata4.posterior[name]))
       for name in param_names]
```

The following table summarizes the results.

```
index = columns[1:]
table = pd.DataFrame(index=index)
table['Posterior mean'] = np.round(means, 3)
table['94% CI'] = cis
table
```

| | Posterior mean | 94% CI |
|---|---|---|
| **Logged GDP per capita** | 0.250 | [0.088, 0.413] |
| **Social support** | 0.223 | [0.076, 0.374] |
| **Healthy life expectancy** | 0.223 | [0.076, 0.374] |
| **Freedom to make life choices** | 0.188 | [0.087, 0.285] |
| **Generosity** | 0.054 | [-0.029, 0.14] |
| **Perceptions of corruption** | -0.100 | [-0.197, -0.006] |

It looks like GDP has the strongest association with happiness (or satisfaction), followed by social support, life expectancy, and freedom.

After controlling for those other factors, the parameters of the other factors are substantially smaller, and since the CI for generosity includes 0, it is plausible that generosity is not substantially related to happiness, at least as they were measured in this study.

This example demonstrates the power of MCMC to handle models with more than a few parameters. But it does not really demonstrate the power of Bayesian regression.

If the goal of a regression model is to estimate parameters, there is no great advantage to Bayesian regression compared to conventional least squares regression.

Bayesian methods are more useful if we plan to use the posterior distribution of the parameters as part of a decision analysis process.

## Summary

In this chapter we used PyMC to implement two models we've seen before: a Poisson model of goal-scoring in soccer and a simple regression model. Then we implemented a multiple regression model that would not have been possible to compute with a grid approximation.

MCMC is more powerful than grid methods, but that power comes with some disadvantages:

- MCMC algorithms are fiddly. The same model might behave well with some priors and less well with others. And the sampling process often produces warnings about tuning steps, divergences, "r-hat statistics", acceptance rates, and effective samples. It takes some expertise to diagnose and correct these issues.
- I find it easier to develop models incrementally using grid algorithms, checking intermediate results along the way. With PyMC, it is not as easy to be confident that you have specified a model correctly.

For these reasons, I recommend a model development process that starts with grid algorithms and resorts to MCMC if necessary. As we saw in the previous chapters, you can solve a lot of real-world problems with grid methods. But when you need MCMC, it is useful to have a grid algorithm to compare to (even if it is based on a simpler model).

All of the models in this book can be implemented in PyMC, but some of them are easier to translate than others. In the exercises, you will have a chance to practice.

## Exercises

**Exercise:** As a warmup, let's use PyMC to solve the Euro problem. Suppose we spin a coin 250 times and it comes up heads 140 times. What is the posterior distribution of $x$, the probability of heads?

For the prior, use a beta distribution with parameters $\alpha = 1$ and $\beta = 1$.

See the PyMC documentation for the list of continuous distributions.

```
# Solution

n = 250
k_obs = 140

with pm.Model() as model5:
    x = pm.Beta('x', alpha=1, beta=1)
    k = pm.Binomial('k', n=n, p=x, observed=k_obs)
    idata5 = pm.sample(500, **options)
    az.plot_posterior(idata5)
```

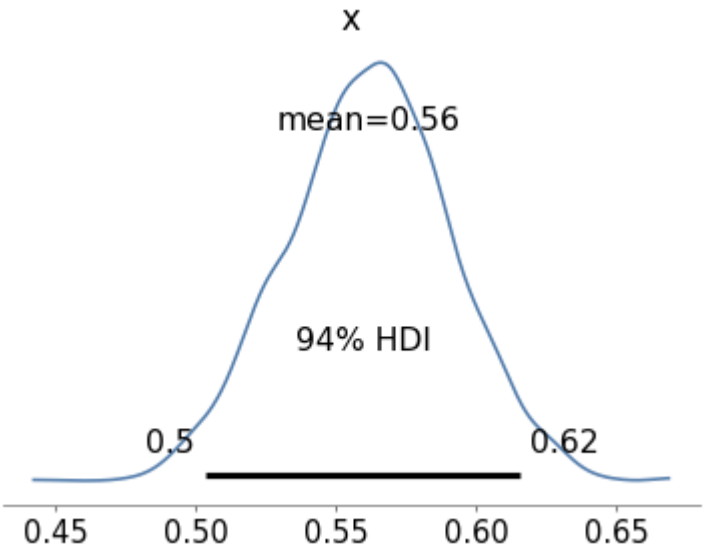
```
Initializing NUTS using jitter+adapt_diag...
```

```
Multiprocess sampling (4 chains in 4 jobs)
```

```
NUTS: [x]
```

```
/Users/anton/.pyenv/versions/3.14.3/lib/python3.14/site-packages/rich/live.py:260: UserWarning:
"ipywidgets" for Jupyter support
  warnings.warn('install "ipywidgets" for Jupyter support')
```

```
Sampling 4 chains for 1_000 tune and 500 draw iterations (4_000 + 2_000 draws total) took 0 sec
onds.
```




**Exercise:** Now let's use PyMC to replicate the solution to the Grizzly Bear problem in <<_TheGrizzlyBearProblem>>, which is based on the hypergeometric distribution.

I'll present the problem with slightly different notation, to make it consistent with PyMC.

Suppose that during the first session, `k=23` bears are tagged. During the second session, `n=19` bears are identified, of which `x=4` had been tagged.

Estimate the posterior distribution of `N`, the number of bears in the environment.

For the prior, use a discrete uniform distribution from 50 to 500.

See the PyMC documentation for the list of discrete distributions.

Note: `HyperGeometric` was added to PyMC after version 3.8, so you might need to update your installation to do this exercise.

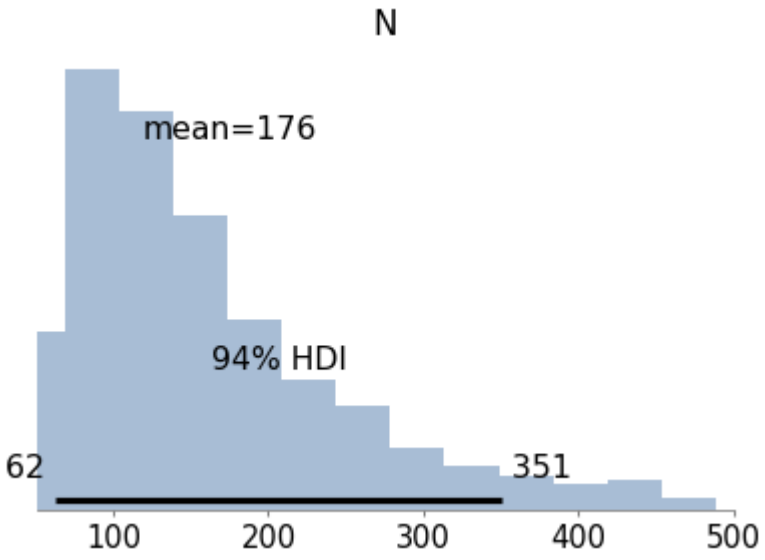
```
# Solution

k = 23
n = 19
x = 4

with pm.Model() as model6:
    N = pm.DiscreteUniform('N', 50, 500)
    y = pm.HyperGeometric('y', N=N, k=k, n=n, observed=x)
    idata6 = pm.sample(1000, **options)
    az.plot_posterior(idata6)
```

```
Multiprocess sampling (4 chains in 4 jobs)
```

```
Metropolis: [N]
```

```
/Users/anton/.pyenv/versions/3.14.3/lib/python3.14/site-packages/rich/live.py:260: UserWarning:
"ipywidgets" for Jupyter support
  warnings.warn('install "ipywidgets" for Jupyter support')
```

```
Sampling 4 chains for 1_000 tune and 1_000 draw iterations (4_000 + 4_000 draws total) took 0 s
econds.
```




**Exercise:** In <<_TheWeibullDistribution>> we generated a sample from a Weibull distribution with $\lambda = 3$ and $k = 0.8$. Then we used the data to compute a grid approximation of the posterior distribution of those parameters.

Now let's do the same with PyMC.

For the priors, you can use uniform distributions as we did in <<_SurvivalAnalysis>>, or you could use `HalfNormal` distributions provided by PyMC.

Note: The `Weibull` class in PyMC uses different parameters than SciPy. The parameter `alpha` in PyMC corresponds to $k$, and `beta` corresponds to $\lambda$.

Here's the data again:

```
data = [0.80497283, 2.11577082, 0.43308797, 0.10862644, 5.17334866,
        3.25745053, 3.05555883, 2.47401062, 0.05340806, 1.08386395]
```

```
# Solution

with pm.Model() as model7:
    lam = pm.Uniform('lam', 0.1, 10.1)
    k = pm.Uniform('k', 0.1, 5.1)
    y = pm.Weibull('y', alpha=k, beta=lam, observed=data)
    idata7 = pm.sample(1000, **options)
    az.plot_posterior(idata7)
```

```
Initializing NUTS using jitter+adapt_diag...
```

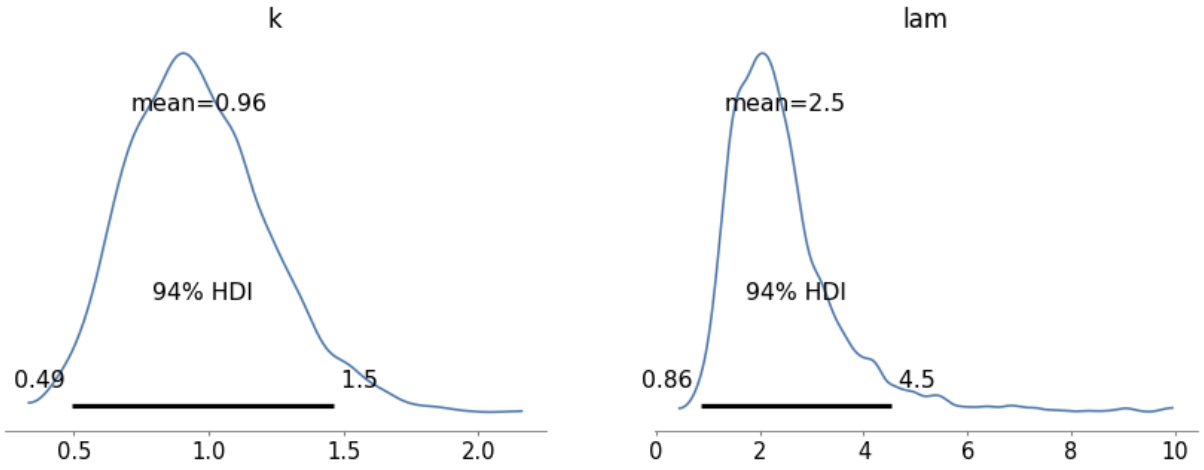
```
Multiprocess sampling (4 chains in 4 jobs)
```

```
NUTS: [lam, k]
```

```
/Users/anton/.pyenv/versions/3.14.3/lib/python3.14/site-packages/rich/live.py:260: UserWarning:
"ipywidgets" for Jupyter support
  warnings.warn('install "ipywidgets" for Jupyter support')
```

```
Sampling 4 chains for 1_000 tune and 1_000 draw iterations (4_000 + 4_000 draws total) took 0 s
econds.
```

```
There was 1 divergence after tuning. Increase `target_accept` or reparameterize.
```




**Exercise:** In <<_ImprovingReadingAbility>> we used data from a reading test to estimate the parameters of a normal distribution.

Make a model that defines uniform prior distributions for `mu` and `sigma` and uses the data to estimate their posterior distributions.

Here's the data again.

```
download('https://github.com/AllenDowney/ThinkBayes2/raw/master/data/drp_scores.csv')
```

```
import pandas as pd

df = pd.read_csv('drp_scores.csv', skiprows=21, delimiter='\t')
df.head()
```

| | Treatment | Response |
|---|---|---|
| 0 | Treated | 24 |
| 1 | Treated | 43 |
| 2 | Treated | 58 |
| 3 | Treated | 71 |
| 4 | Treated | 43 |

I'll use `groupby` to separate the treated group from the control group.

```
grouped = df.groupby('Treatment')
responses = {}

for name, group in grouped:
    responses[name] = group['Response']
```

Now estimate the parameters for the treated group.

```
data = responses['Treated']
```

```
# Solution

with pm.Model() as model8:
    mu = pm.Uniform('mu', 20, 80)
    sigma = pm.Uniform('sigma', 5, 30)
    y = pm.Normal('y', mu, sigma, observed=data)
    idata8 = pm.sample(500, **options)
```

```
Initializing NUTS using jitter+adapt_diag...
```

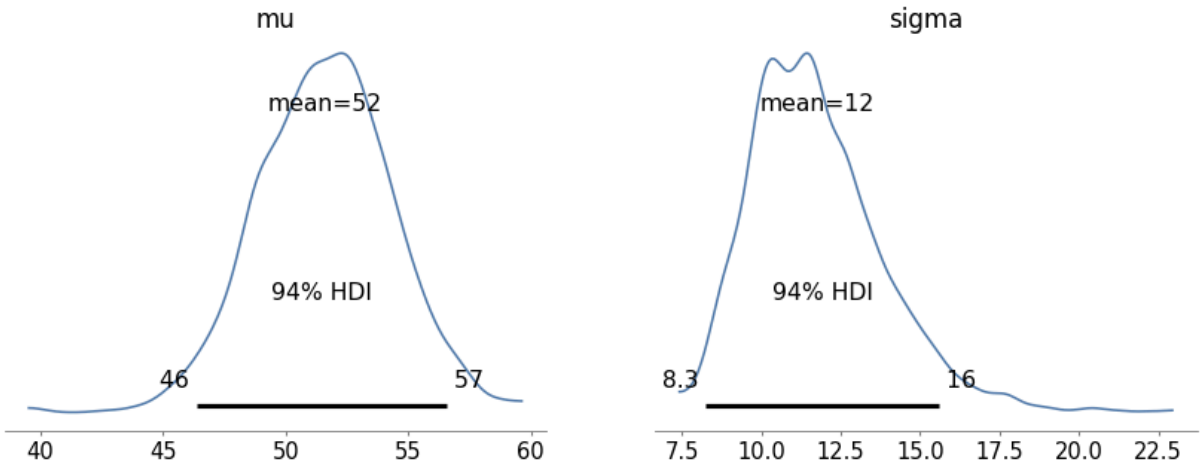
```
Multiprocess sampling (4 chains in 4 jobs)
```

```
NUTS: [mu, sigma]
```

```
/Users/anton/.pyenv/versions/3.14.3/lib/python3.14/site-packages/rich/live.py:260: UserWarning:
"ipywidgets" for Jupyter support
  warnings.warn('install "ipywidgets" for Jupyter support')
```

```
Sampling 4 chains for 1_000 tune and 500 draw iterations (4_000 + 2_000 draws total) took 0 sec
onds.
```

```
# Solution

with model8:
    az.plot_posterior(idata8)
```




**Exercise:** In <<_TheLincolnIndexProblem>> we used a grid algorithm to solve the Lincoln Index problem as presented by John D. Cook:

> *"Suppose you have a tester who finds 20 bugs in your program. You want to estimate how many bugs are really in the program. You know there are at least 20 bugs, and if you have supreme confidence in your tester, you may suppose there are around 20 bugs. But maybe your tester isn't very good. Maybe there are hundreds of bugs. How can you have any idea how many bugs there are? There's no way to know with one tester. But if you have two testers, you can get a good idea, even if you don't know how skilled the testers are."*

Suppose the first tester finds 20 bugs, the second finds 15, and they find 3 in common; use PyMC to estimate the number of bugs.

I'll use the following notation for the data:

- k11 is the number of bugs found by both testers,
- k10 is the number of bugs found by the first tester but not the second,
- k01 is the number of bugs found by the second tester but not the first, and
- k00 is the unknown number of undiscovered bugs.

Here are the values for all but `k00`:

```
k10 = 20 - 3
k01 = 15 - 3
k11 = 3
```

In total, 32 bugs have been discovered:

```
k_obs = [k01, k10, k11]
n_obs = np.sum(k_obs)
n_obs
```

```
np.int64(32)
```

Note: This exercise is more difficult that some of the previous ones, and the solution that appeared in previous versions of this notebook doesn't work in recent versions of PyMC. Here are some suggestions that might help.

- Define priors for `p0`, `p1`, and `N`.
- Compute the complementary probabilities, `q1` and `q2`, well as the probability of being seen, `p_seen`, and the probabilities of being seen by the first, second, or both testers, given that a bug is discovered.

```
q0 = 1-p0
q1 = 1-p1
p_seen = 1 - q0*q1
ps = pm.math.stack([q0*p1, p0*q1, p0*p1]) / p_seen
```

- Compute *two* likelihoods: the probability of finding `n_obs` bugs given `N` and `p_seen`, and the probability of `k_obs` given `n_obs` and `ps`.

```
# Solution

with pm.Model() as model9:
    p0 = pm.Beta('p0', alpha=1, beta=1)
    p1 = pm.Beta('p1', alpha=1, beta=1)
    N = pm.DiscreteUniform('N', n_obs, 350)

    q0 = 1-p0
    q1 = 1-p1
    p_seen = 1 - q0*q1
    ps = pm.math.stack([q0*p1, p0*q1, p0*p1]) / p_seen

    pm.Binomial("n_obs", n=N, p=p_seen, observed=n_obs)
    pm.Multinomial('k_obs', n=n_obs, p=ps, observed=k_obs)
```

```
# Solution

with model9:
    idata9 = pm.sample(1000)
```

```
Multiprocess sampling (4 chains in 4 jobs)
```

```
CompoundStep
```

```
>NUTS: [p0, p1]
```

```
>Metropolis: [N]
```

```
/Users/anton/.pyenv/versions/3.14.3/lib/python3.14/site-packages/rich/live.py:260: UserWarning:
"ipywidgets" for Jupyter support
  warnings.warn('install "ipywidgets" for Jupyter support')
```

```
Sampling 4 chains for 1_000 tune and 1_000 draw iterations (4_000 + 4_000 draws total) took 0 s
econds.
```

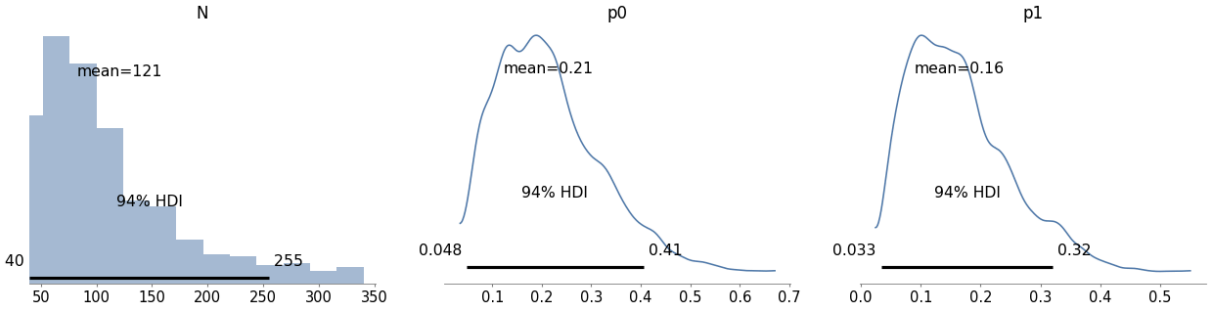
```
The rhat statistic is larger than 1.01 for some parameters. This indicates problems during samp
ling. See https://arxiv.org/abs/1903.08008 for details
```

```
The effective sample size per chain is smaller than 100 for some parameters.  A higher number i
s needed for reliable rhat and ess computation. See https://arxiv.org/abs/1903.08008 for detail
s
```

```
# Solution

az.summary(idata9)
```

| | mean | sd | hdi_3% | hdi_97% | mcse_mean | mcse_sd | e |
|---|---|---|---|---|---|---|---|
| **N** | 121.324 | 65.575 | 40.000 | 255.000 | 15.047 | 10.807 | |
| **p0** | 0.213 | 0.104 | 0.048 | 0.405 | 0.021 | 0.015 | |
| **p1** | 0.164 | 0.084 | 0.033 | 0.321 | 0.016 | 0.011 | |

```
# Solution

with model9:
    az.plot_posterior(idata9)
```




*Think Bayes*, Second Edition